

DINEMATCH

RESTAURANT RECOMMENDATION PLATFORM

Project Deliverable 2

1. PROJECT TITLE

DINEMATCH

A PERSONALIZED RESTAURANT RECOMMENDATION PLATFORM

2. TASK DELEGATION

FATHIMA BINT-KHALID – FXB210001

PROJECT COORDINATOR

Report writing and formatting, project scheduling, comparison with similar designs, conclusion, references, presentation slides. Report sections: 4a, 6, 7, 8

UMAR A. HASAN -

SYSTEMS ARCHITECT

Deliverable #1 content recap, architectural design updates, UI prototype in Figma. Report sections: 3.1-3.9, 9 (architecture), prototype

SAUMYA ADITYA PATEL -

SOFTWARE ENGINEER & QA LEAD

Test plan design and execution, unit test code and automated testing, GitHub repository management. Report sections: 5

DAVID HUYNH -

FRONT END & UX, UI

Cost, effort, and pricing estimation using FP Analysis, hardware/software/personnel cost estimation. Report sections: 4b, 4c-4e

3. DELIVERABLE #1 CONTENT (REPRODUCED)

3.1 PROPOSAL FEEDBACK

The feedback we received on our proposal was that the project plan appears to be sound. We're glad it was approved and are moving forward as proposed

3.2 SOFTWARE PROCESS MODEL

We're using Scrum for DineMatch because our requirements might change as we go, if we need to tweak the recommendation engine or add filters. Scrum lets us work in sprints so we can build and test one thing at a time. We use a team group chat for check-ins and to manage our task list. Since we're a small team, this felt better than Waterfall where you plan everything upfront.

3.3 SOFTWARE REQUIREMENTS

FUNCTIONAL REQUIREMENTS

- FR1: Users can filter restaurants by cuisine, price, location, and vibe.
- FR2: The system ranks restaurants by a relevance score based on the user's filters.
- FR3: Users can create an account and log in using authentication.
- FR4: Logged-in users can bookmark restaurants and view their saved list.
- FR5: Logged-in users can leave a star rating and text review on any restaurant.
- FR6: Restaurant search results include an embedded Google Maps view.
- FR7: The system suggests restaurants based on a user's past searches and bookmarks.

NON-FUNCTIONAL REQUIREMENTS

PRODUCT REQUIREMENTS:

- Performance: Search results load in under 3 seconds.
- Space: The app uses no more than 500MB of server storage.
- Usability: A new user can search for a restaurant without any tutorial.
- Dependability: The system stays up 99% of the time during peak hours.
- Security: Passwords are stored securely and tokens expire after 24 hours.

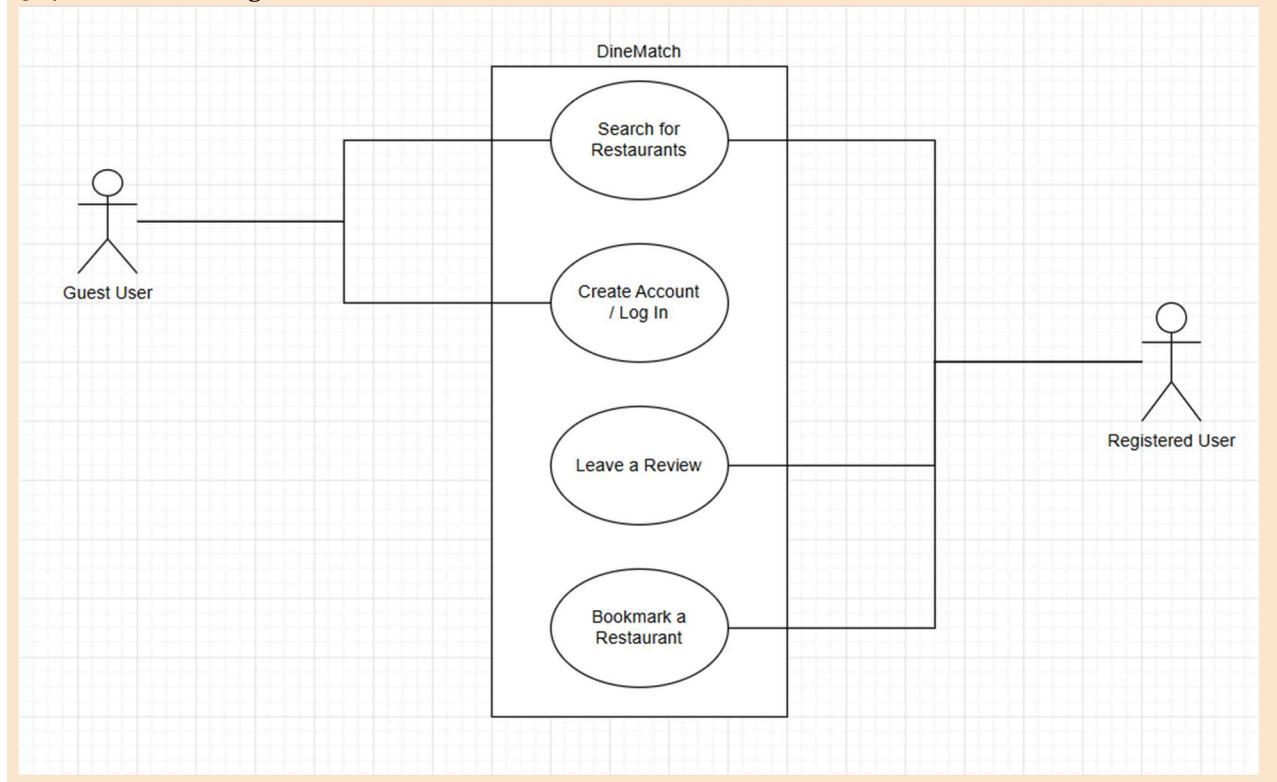
ORGANIZATIONAL REQUIREMENTS:

- Environmental: The app can be deployed on standard cloud infrastructure.
- Operational: The system follows Google Maps API usage guidelines.
- Development: All code changes are committed to the team's GitHub repo

EXTERNAL REQUIREMENTS:

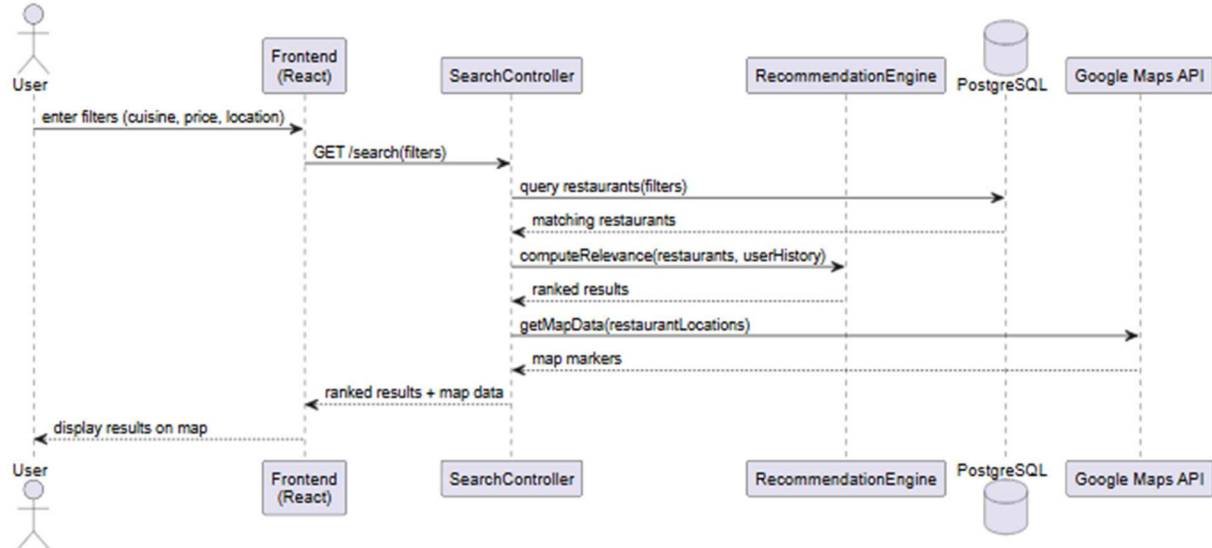
- Accounting: The system logs all bookmarks and review activity.
- Safety/Security: The system is protected against attacks.
- Ethical: User data is only used for recommendations, nothing else.
- Legislative: Users must opt in before their data is used for personalized recommendations

3.4 Use Case Diagram

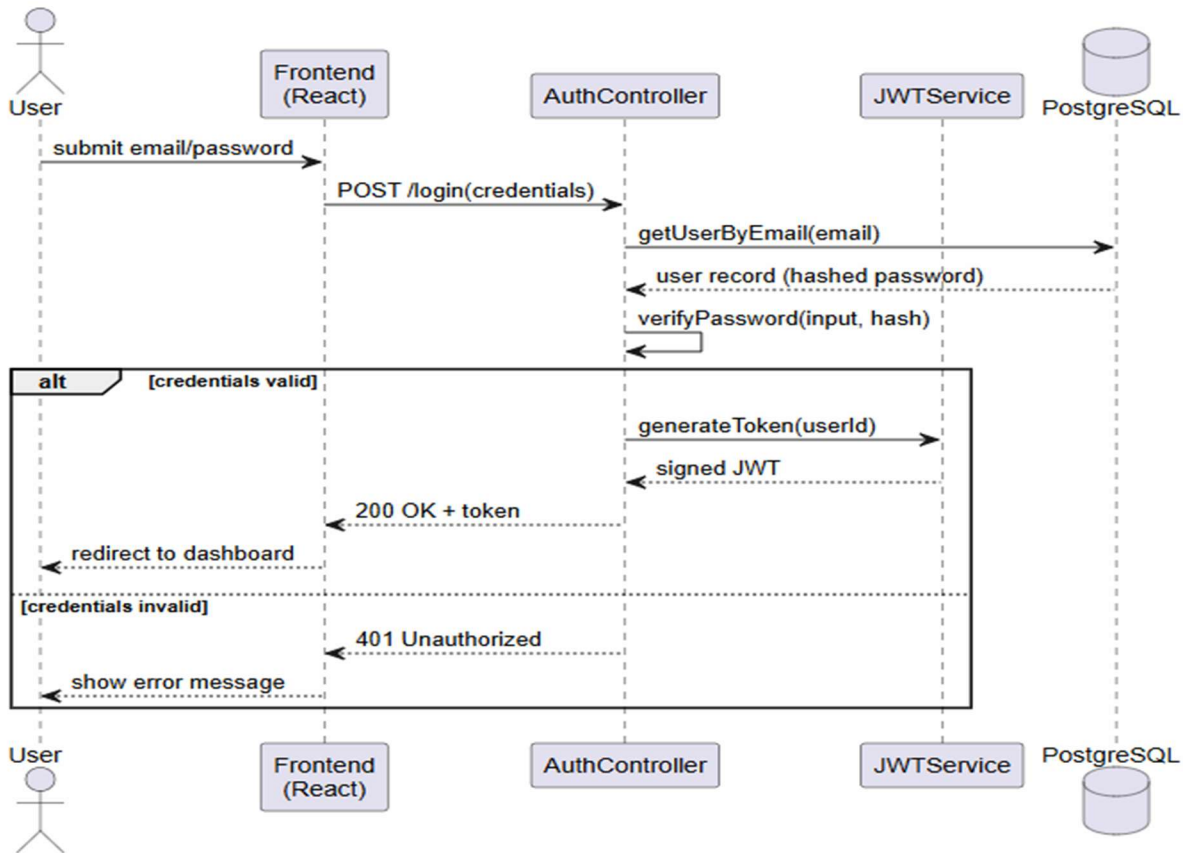


3.5 SEQUENCE DIAGRAMS

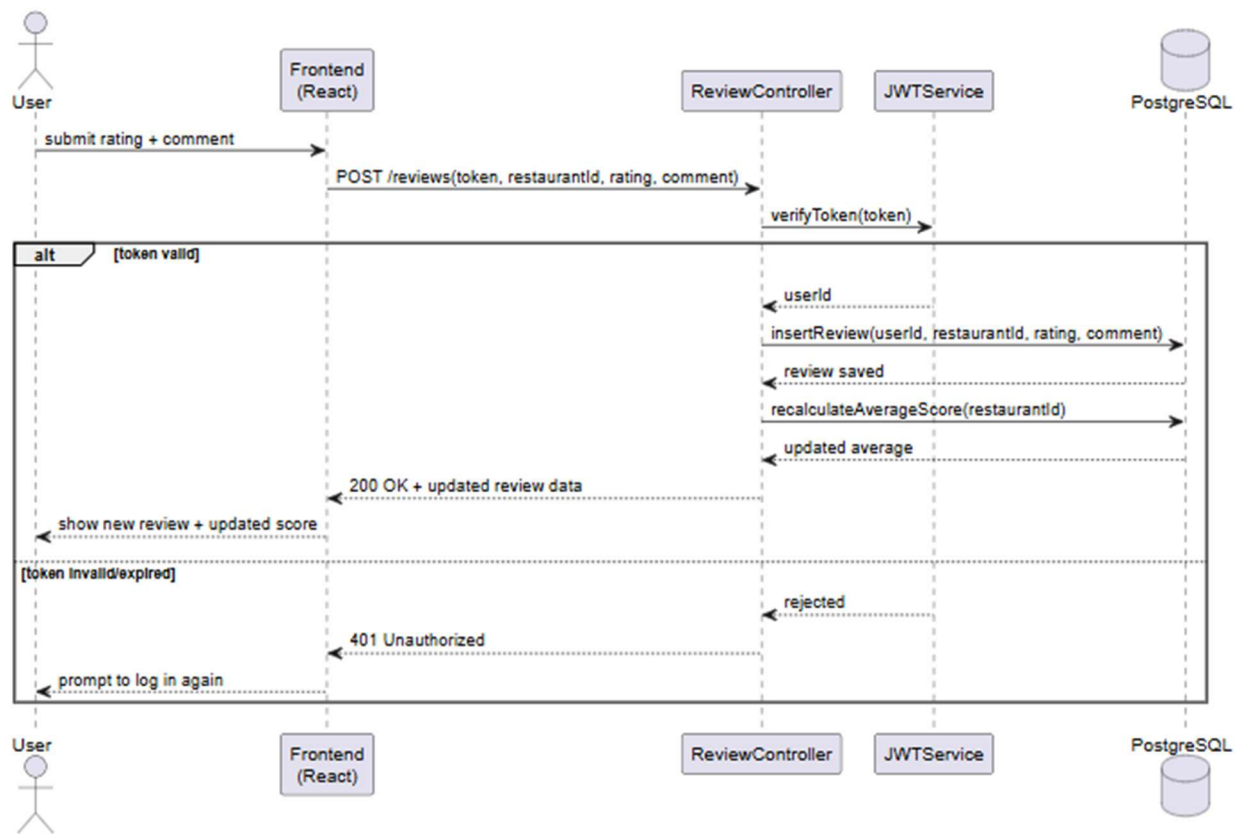
SEARCH FOR RESTURANTS



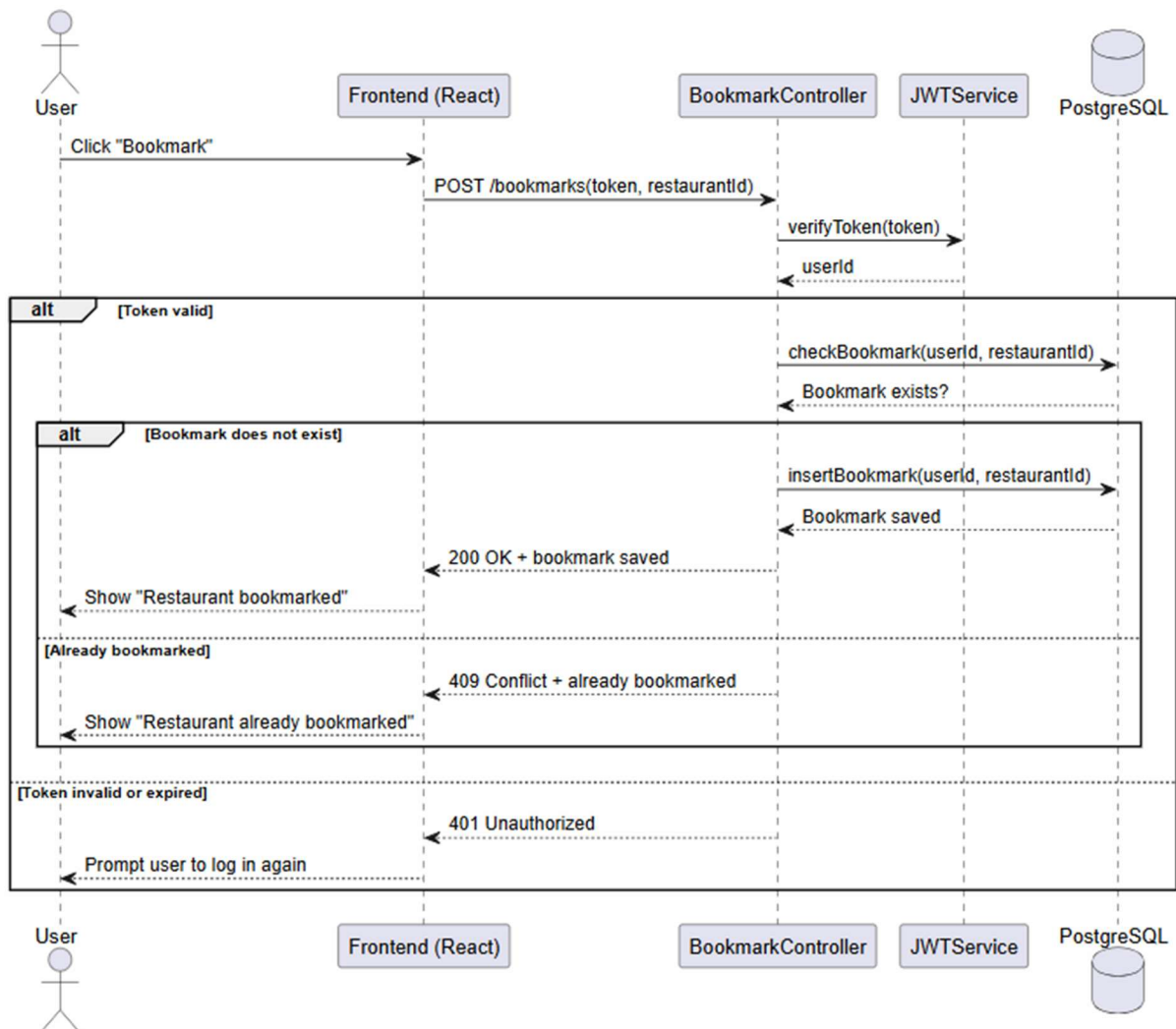
CREATE ACCOUNT / LOG IN



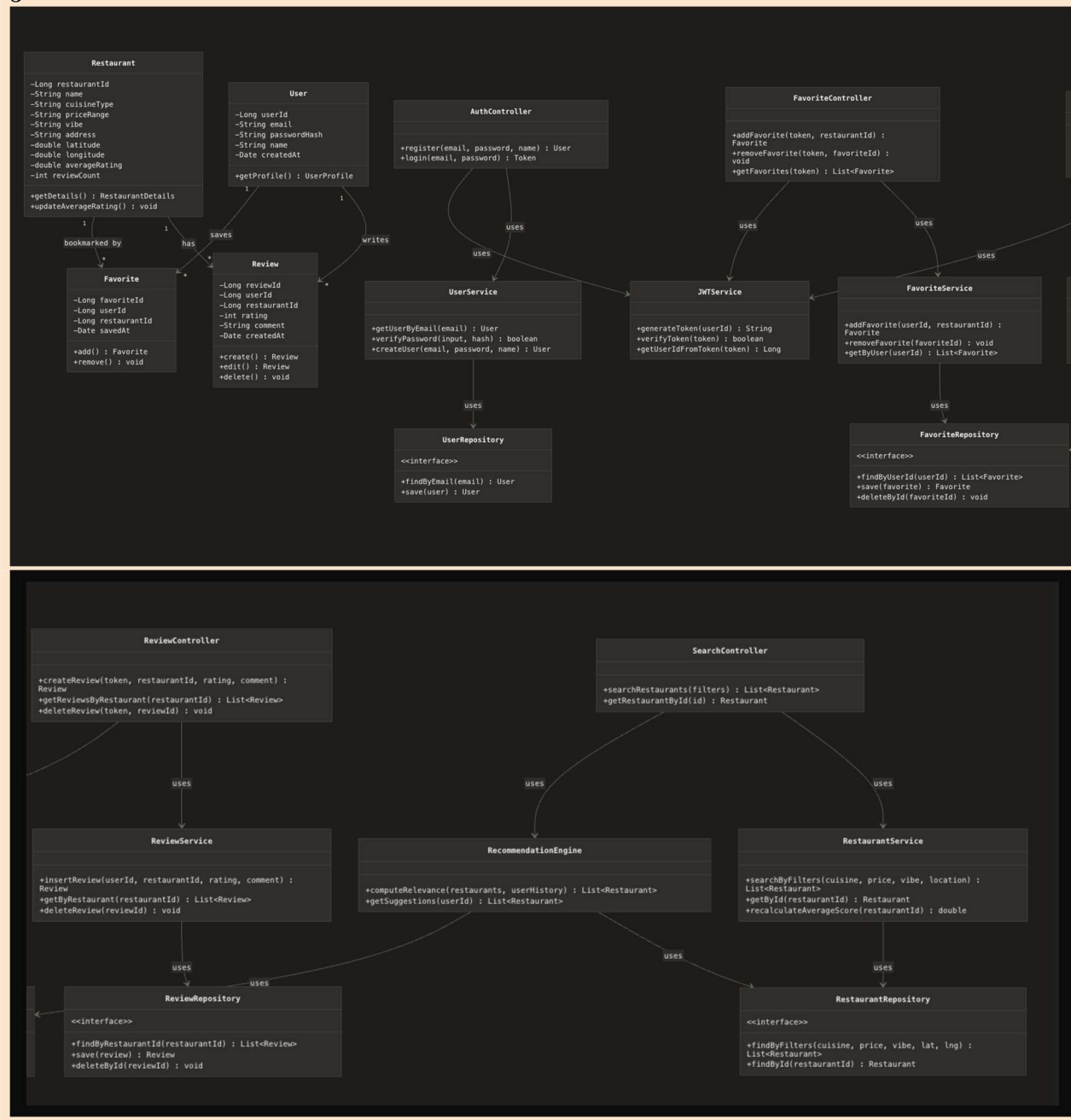
LEAVE A REVIEW



BOOKMARK A RESTAURANT



3.6 CLASS DIAGRAM



3.7 ARCHITECTURAL DESIGN

Our architectural design for our restaurant recommendation platform will be the Model-View-Controller (MVC). This provides a clear separation between application data, user interface (UI), and application logic. This pairs perfectly with our system that requires frequent user input and dynamic content. The Model would be responsible for managing restaurant information, user accounts, ratings, reviews, favorite restaurants, and recommendation data. It will also contain business logic to process searches, calculate recommendations, etc. The View represents the UI which displays restaurant listings, reviews, maps, and more. The Controller will serve as the intermediary between the user and system by processing user requests such as logging in, restaurant searching, and filtering results.

3.8 Restaurant Data Source

Our restaurant data comes from Google Maps Places API. This gives us restaurant names, locations, cuisine type, and price level. Since Google Maps doesn't track "vibe" (like casual or romantic), admins add that info by hand when a new restaurant gets added to DineMatch. This keeps our main data accurate while still letting us add our own custom info.

3.9 UI PROTOTYPE

Link: <https://fork-finder-app.lovable.app/>

This is a fully interactive UX/UI prototype of the app.

3.10 GITHUB REPOSITORY

GitHub: <https://github.com/Saumya-Patel-10/3354-Team5>

4. PROJECT SCHEDULING, COST, EFFORT, AND PRICING ESTIMATION

4A. PROJECT SCHEDULE

Start Date: January 6, 2025

End Date: May 30, 2025

Total Duration: Approximately 5 months (21 weeks)

Justification: DineMatch is a medium-complexity web application with a React frontend, Java/Spring Boot backend, PostgreSQL database, and third-party API integrations (Google Maps). Based on industry benchmarks for comparable restaurant discovery platforms, a 5-month timeline allows for iterative development across two major phases: core features (search, accounts, reviews) in months 1–3, and advanced features (recommendations, bookmarks, map integration) plus testing in months 4–5.

Schedule Parameters:

- Weekends are **not** counted in the working schedule.
- Working hours per day: **8 hours** (9:00 AM – 5:00 PM).
- Total working days: approximately **104 working days** (21 weeks × 5 days/week, minus holidays).

Phase Breakdown:

Phase	Duration	Dates	Key Activities
Planning & Design	3 weeks	Jan 6 – Jan 24, 2025	Requirements finalization, UI wireframes, database schema design, architecture decisions
Sprint 1: Core Backend	4 weeks	Jan 27 – Feb 21, 2025	User authentication (JWT), restaurant data models, PostgreSQL setup, REST API endpoints
Sprint 2: Core Frontend	4 weeks	Feb 24 – Mar 21, 2025	React component architecture, search UI, filter components, Google Maps integration
Sprint 3: Advanced Features	4 weeks	Mar 24 – Apr 18, 2025	Recommendation engine, bookmarks, reviews, user dashboard
Sprint 4: Testing & QA	3 weeks	Apr 21 – May 9, 2025	Unit testing, integration testing, performance optimization, bug fixes
Deployment & Launch	3 weeks	May 12 – May 30, 2025	Cloud deployment, final QA, documentation, user acceptance testing

4B. COST, EFFORT, AND PRICING ESTIMATION

We selected **Function Point (FP) Analysis** as our cost modeling technique because DineMatch is a user-facing web application where the complexity is best measured by its functional components (inputs, outputs, queries, files, and interfaces) rather than by lines of code.

STEP 1: IDENTIFY AND CLASSIFY FUNCTION POINTS

EXTERNAL INPUTS (EI):

Transactions where data crosses the boundary into the system.

EI	Complexity	Weight
User Registration (name, email, password)	Low	3
User Login (email, password)	Low	3
Submit Restaurant Review (rating, comment, restaurantId)	Average	4
Bookmark a Restaurant (restaurantId)	Low	3
Search/Filter Restaurants (cuisine, price, location, vibe)	High	6
Subtotal EI		19

EXTERNAL OUTPUTS (EO):

Transactions where derived data crosses the boundary out of the system.

EO	Complexity	Weight
Ranked Search Results Display	High	7
Personalized Recommendation List	High	7
Restaurant Detail View (info + reviews + map)	Average	5
User Dashboard (bookmarks, search history)	Average	5
Subtotal EO		24

EXTERNAL INQUIRIES (EQ):

Input/output combinations that retrieve data without modifying it.

EQ	Complexity	Weight
View Bookmarked Restaurants	Low	3
View User Profile	Low	3
View Restaurant Reviews	Average	4
Subtotal EQ		10

INTERNAL LOGICAL FILES (ILF):

User-identifiable groups of logically related data maintained within the system.

ILF	Complexity	Weight
Users	Low	7
Restaurants	High	15
Reviews	Average	10
Bookmarks	Low	7
Search/Recommendation History	Average	10
Subtotal ILF		49

EXTERNAL INTERFACE FILES (EIF):

Data referenced by the system but maintained externally.

EIF	Complexity	Weight
Google Maps API	High	10
Subtotal EIF		10

Unadjusted Function Points (UFP) = 19 + 24 + 10 + 49 + 10 = 112

STEP 2: VALUE ADJUSTMENT FACTOR (VAF)

The 14 General System Characteristics (GSC) with each rated on a scale of 0 (no influence) to 5 (strong influence):

#	<i>General System Characteristic</i>	<i>Rating</i>	<i>Justification</i>
1	Data Communications	4	Web-based with REST API calls and third-party API
2	Distributed Data Processing	2	Standard client-server; no complex distribution
3	Performance	3	3-second load time requirement for searches
4	Heavily Used Configuration	2	Runs on standard cloud infrastructure
5	Transaction Rate	3	Moderate concurrent searches and reviews
6	Online Data Entry	5	All user input is online (search, register, review)
7	End-User Efficiency	4	UI-focused with filters, maps, and recommendations
8	Online Update	3	Real-time review and bookmark updates
9	Complex Processing	3	Recommendation engine with relevance scoring
10	Reusability	2	Some reusable React components and API services
11	Installation Ease	3	Cloud deployment via containerization
12	Operational Ease	4	Minimal admin required after deployment
13	Multiple Sites	3	Cloud-hosted, accessible from multiple locations
14	Facilitate Change	3	Modular MVC design supports future extensions

Total Degree of Influence (TDI) = 4 + 2 + 3 + 2 + 3 + 5 + 4 + 3 + 3 + 2 + 3 + 4 + 3 + 3 = 44

VAF = (TDI × 0.01) + 0.65 = (44 × 0.01) + 0.65 = 1.09

STEP 3: ADJUSTED FUNCTION POINTS

AFP = UFP × VAF = 112 × 1.09 = 122.08 ≈ 122 Adjusted Function Points

STEP 4: EFFORT ESTIMATION

Using an industry-average productivity rate of 10 Function Points per person-month for web applications [1]:

Effort = AFP / Productivity = 122 / 10 = 12.2 person-months

Converting to hours (at 160 working hours per month):

Total Effort = 12.2 × 160 = 1,952 person-hours

STEP 5: COST AND PRICING

Using a blended developer billing rate of \$75/hour:

Development Cost = 1,952 hours × \$75/hour = \$146,400

ADDING HARDWARE, SOFTWARE, AND PERSONNEL OVERHEAD

(calculated in Sections 4c–4e):

Cost Category	Estimated Cost
Development Labor (FP-based)	\$146,400
Hardware	\$12,300
Software Licenses & Services	\$5,521
Additional Personnel (PM + Training)	\$38,333
Total Project Cost	\$202,554

Applying a 20% profit margin for pricing:

Project Price = \$202,554 × 1.20 = \$243,065

4C. ESTIMATED COST OF HARDWARE PRODUCTS

Hardware Item	Quantity	Unit Cost	Duration	Total Cost
AWS EC2 Instance (t3.medium) — application server	2	\$75/month	12 months	\$1,800
AWS RDS Instance (db.t3.medium) — PostgreSQL	1	\$100/month	12 months	\$1,200
AWS S3 — static asset storage	1	\$25/month	12 months	\$300
Development Workstations (laptops)	6	\$1,500 each	One-time	\$9,000
Total Hardware Cost				\$12,300

Notes: Cloud server costs are estimated for the first year of operation. Development workstations are a one-time capital expense for each team member. Costs are based on AWS pricing as of 2025 [2].

4D. ESTIMATED COST OF SOFTWARE PRODUCTS

Software Product	Quantity	Unit Cost	Duration	Total Cost
Google Maps Platform API (Maps + Places)	1	\$200/month	12 months	\$2,400
GitHub Team Plan	6 users	\$4/user/month	12 months	\$288
JetBrains IntelliJ IDEA Ultimate	5 licenses	\$499/license	Annual	\$2,495
Figma Professional (UI design)	2 seats	\$12/seat/month	12 months	\$288
Domain Registration + SSL Certificate	1	\$50/year	Annual	\$50
React, Spring Boot, PostgreSQL, Node.js	—	Free (open source)	—	\$0
Total Software Cost				\$5,521

Notes: Google Maps API cost assumes moderate usage under the standard pricing tier. Open-source technologies (React, Spring Boot, PostgreSQL) incur no licensing costs [3].

4E. ESTIMATED COST OF PERSONNEL

5-MONTH DEVELOPMENT CYCLE:

Role	Headcount	Annual Salary	Monthly Cost	Duration	Total Cost
Frontend Developer (React)	2	\$85,000	\$7,083	5 months	\$70,833
Backend Developer (Java/Spring)	2	\$90,000	\$7,500	5 months	\$75,000
Full-Stack / DevOps Engineer	1	\$95,000	\$7,917	5 months	\$39,583
Project Manager	1	\$80,000	\$6,667	5 months	\$33,333
Subtotal (Development Personnel)	6				\$218,749

INSTALLATION TRAINING COST:

Training Activity	Cost
End-user documentation and video tutorials	\$2,000
Administrator training (deployment, monitoring)	\$3,000
Subtotal Training	\$5,000

Total Personnel Cost: \$223,749

Note on FP vs. Personnel overlap: The Function Point–based development cost in Section 4b (\$146,400) estimates labor using an industry billing rate. The personnel table above reflects actual salary-based costs if the project were staffed internally. In practice, a company would use one model or the other; we present both for completeness, and the total project cost in Section 4b uses the FP-derived development labor figure plus the project manager and training costs as overhead (\$38,333).

5. TEST PLAN

5a. What We Tested

5B. CODE

5c. Test Results

6. Comparison with Similar Apps

I think DineMatch stands out from other restaurant apps in a few ways.

Yelp has reviews and basic filters, but it doesn't personalize rankings. Everyone sees the same order based on review count. DineMatch ranks results based on each user's own history.

Google Maps is great for directions and location info, but it doesn't let you filter by vibe or save and review restaurants in a focused way. We use Google Maps for the map part but build the dining experience on top of it.

TripAdvisor covers a lot of cities, but it shows the same ranking to everyone. DineMatch's scores are personal to each user.

Because of this, we decided to build our own recommendation engine, use Google Maps just for maps and data instead of building our own, keep our features focused, and use free open-source tools to save money.

7. Conclusion

I think DineMatch fills a real gap. Most apps show restaurants but don't personalize results. We built out the requirements, diagrams, architecture, a cost estimate, and a tested recommendation engine.

We changed two things from our original plan:

1. Testing tool: we switched from JUnit to Jest since our scoring logic is shared with React. I think this was fine since Jest tests just as well.
2. Cost method: we used Function Points instead of Application Composition to get a more detailed cost estimate.

8. References

- [1] R. S. Pressman and B. R. Maxim, Software Engineering: A Practitioner's Approach, 9th ed. McGraw Hill, 2020.
- [2] Amazon Web Services, "Amazon EC2 Pricing," 2025.
- [3] Google, "Google Maps Platform Pricing," 2025.
- [4] Yelp Inc., "Yelp for Business," 2025.
- [5] Google, "Google Maps," 2025.

- [6] Tripadvisor LLC, "Tripadvisor," 2025.
- [7] I. Sommerville, Software Engineering, 10th ed. Pearson, 2016.
- [8] Lovable, "Lovable Project," Lovable, 2026. [Online]. Available:
<https://lovable.dev/projects/03c2d5e8-5e53-4e46-a914-ob984cbe63dd>